

Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

ARCHITECTURE FUNDAMENTALS

Communication Architectures

(Patterns)

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 08.2

UCT Unicast (one-to-one)

Communicate messages from one source to exactly one destination node. The destination node is explicitly and individually addressed.

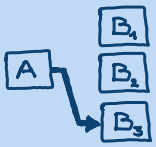
Rationale: private communication between exactly two nodes which both know each other beforehand.



ACT Anycast (one-to-any)

Communicate messages from one source to one of many destination nodes. The picked destination node usually is the network-topology-wise "nearest" or least utilized node in a group of nodes.

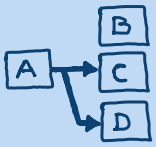
Rationale: Unicast, optimized for network failover scenarios, load balancing and CDNs.



MCT Multicast (one-to-many)

Communicate messages from one source to many destination nodes. The destination nodes usually form a group and are usually not individually addressed.

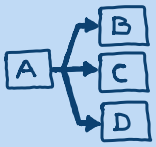
Rationale: node communication where destination nodes dynamically change or where total traffic should be reduced.



BCT Broadcast (one-to-all)

Communicate messages from one source to all available destination nodes. The destination nodes usually are implicitly defined by the extend of the local communication network segment.

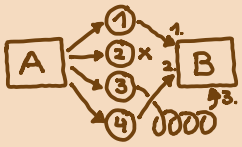
Rationale: spreading out messages to all available nodes for potential responses.



DGR Datagram (Single Packet)

Communicate messages as an unordered set of single packets, usually without any network congestion control, retries or other delivery guarantees.

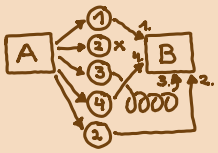
Rationale: simple low-overhead communication without prior communication establishment (handshake).



STR Stream (Sequence of Packets)

Communicate messages as an ordered sequence (stream) of packets, usually with network congestion control, retries and delivery guarantees (at-most-once, exactly-once, at-least-once).

Rationale: reliable communication between nodes.



PLL Pull (Request/Response, RPC)

Communicate by performing a request (from the client node) and pulling a corresponding response (from the server node).

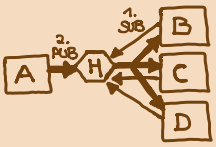
Rationale: Remote Procedure Call (RPC) like Unicast or Anycast communication.



PSH Push (Publish/Subscribe, Events)

Communicate by "subscribing" to "channels" of messages (on one or more receiver nodes or on an intermediate hub) once and then publishing events to those "channels" (on the sender node) multiple times.

Rationale: event-based Multicast or Broadcast communication.



Freiknecht, Content Version 1.0.3 (2021-12-28), Authored 2016/2020 by Dr. Ralf S. Engelschall, Graphical Illustration Version 1.0.1 (2016-11-16), Copyright © 2018 Dr. Ralf S. Engelschall <http://engelschall.com> – All Rights Reserved. Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Bei den **Communication Architectures** wird die Kommunikationsart zwischen Komponenten adressiert. Man unterscheidet primär vier verschiedene Arten des Nachrichtenversands: beim **Unicast** sendet ein Quellknoten an exakt einen direkt adressierten Zielknoten. Beim **Anycast** sendet ein Quellknoten an eine Gruppe von potenziellen Zielknoten, die Nachricht wird aber nur an einen Zielknoten aus der Gruppe zugestellt.

Beim **Multicast** sendet ein Quellknoten ebenfalls an eine Gruppe von Zielknoten, die Nachricht wird aber an alle Zielknoten aus der Gruppe zugestellt. Beim **Broadcast** sendet ein Quellknoten an alle erreichbaren Zielknoten, ohne daß diese konkreten Zielknoten dem Quellknoten bekannt sind.

Bei der Art der Nachrichten unterscheidet man zwei Varianten: beim **Datagram** besteht jede Nachricht aus exakt einem Netzwerkpaket und beim Versand mehrerer Nachrichten werden keine Garantien gegeben, ob und in welcher Reihenfolge die Nachrichten beim Zielknoten ankommen werden. Im Gegensatz dazu besteht beim **Stream** eine Nachricht aus einer Sequenz von Netzwerkpaketen und dabei werden verschiedene Garantien gegeben:

Bei Paketstaus auf Zwischenknoten wird beim **Stream** ggf. die Quelle gedrosselt. Bei Paketverlust werden Pakete erneut versandt. Und man erhält ggf. die Kontrolle darüber, ob das Paket am Zielknoten maximal einmal, genau einmal, oder mindestens einmal zugestellt wird.

Man unterscheidet üblicherweise zwei Modi der Client/Server-Kommunikation: beim **Pull**-Modus sendet der Client eine Anfrage und der Server sendet eine Antwort. Der Server kann hierbei insbesondere nicht (ohne vorherige Anfrage) eine Nachricht proaktiv versenden. Beim **Push**-Modus sendet der Client vorab eine Nachricht an den Server, um bestimmte Arten von Nachrichten zu abonnieren. Danach kann jederzeit der Server eine Nachricht an alle dafür abonnierten Clients senden.

Üblicherweise wird **Pull** über **Unicast**/**Anycast** und als **Stream** implementiert, beispielsweise im Protokoll HTTP. **Push** wird dagegen üblicherweise über **Multicast**/**Broadcast** als **Datagram** implementiert, beispielsweise im Protokoll DHCP.

Fragen

- Welches bekannte Web-Protokoll nutzt eine auf **Unicast**, **Stream** und **Pull** basierende Kommunikation?

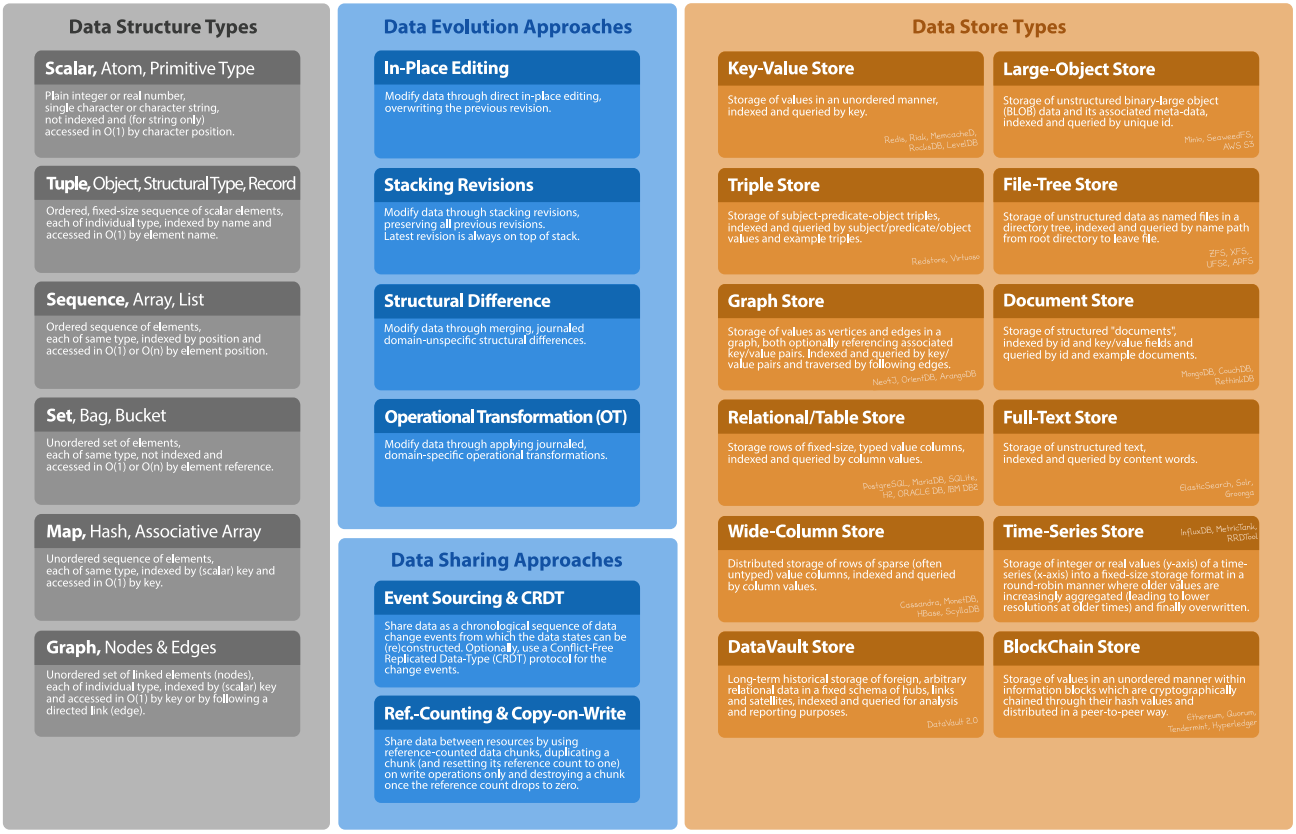
ARCHITECTURE FUNDAMENTALS

Data Structure Architectures

(Patterns)

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 09.1



Intellectual Content: Version 1.1.3 (2021-04-26), Authored 2011-2020 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.1.2 (2021-04-26), Copyright © 2011-2021 Dr. Ralf S. Engelschall <http://engelschall.com>, All rights reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Der Software Architect unterscheidet nur 6 **Data Structure Types** für Daten-Elemente: **Scalar** (z.B. Integer, String, etc.), **Tuple** (ordered fixed-size sequence of Scalars), **Sequence** (ordered sequence of elements), **Set** (unordered set of elements), **Map** (unordered set of elements, each indexed by key) und **Graph** (unordered set of elements, each indexed by key or by following a link between elements). Alle komplexen spezifischen Datenstrukturen in der Praxis sind für den Software Architect nur die Kombination dieser 6 Typen.

Es gibt zahlreiche **Data Evolution Approaches**, mit denen sich Daten über die Zeit verändern können: im einfachsten Fall, **In-Place Editing**, werden Daten einfach direkt geändert. Einen Zugriff auf frühere Stände gibt es hierbei nicht. Soll auf frühere Stände zugegriffen werden können, so kann man **Stacking Revisions** einsetzen, bei dem vor jeder Änderung der gesamte Datensatz zuerst kopiert wird. Damit nicht der gesamte Datensatz kopiert werden muss, wird bei **Structural Difference** nur eine technische Differenz von altem und neuem Datensatz gespeichert. Alternativ können bei **Operational Transformation** die fachlichen Änderungsoperationen als Journal gespeichert werden.

Wird so ein Journal benutzt, um auch Replika der Datensätze aktuell zu halten, spricht man von **Event Sourcing**. Falls das Journal nach dem Protokoll von sogenannten **Conflict-Free Replicated Data-Types (CRDT)** aufgebaut ist, lässt sich statt (unidirektionaler) Replikation auch eine (bidirektionale) Synchronisation erzielen. Falls mehrere konkurrierende Prozesse/Threads logisch auf Kopien, aber physikalisch auf denselben Datensätzen operieren, lässt sich mit **Copy-on-Write** und **Reference Counting** ein gemeinsamer Zugriff und der Lebenszyklus der Datensätze dennoch sinnvoll steuern.

Zur Speicherung von Daten in Datenbanken gibt es zahlreiche **Data Store Types**. Diese unterscheiden sich primär in der Art und Flexibilität der Datenstruktur und den gewährten Garantien. Der verbreitetste Typ ist der **Relational/Table Store**. Der eleganteste Typ ist der **Graph Store**. Der bequemste ist der **Document Store**.

Fragen

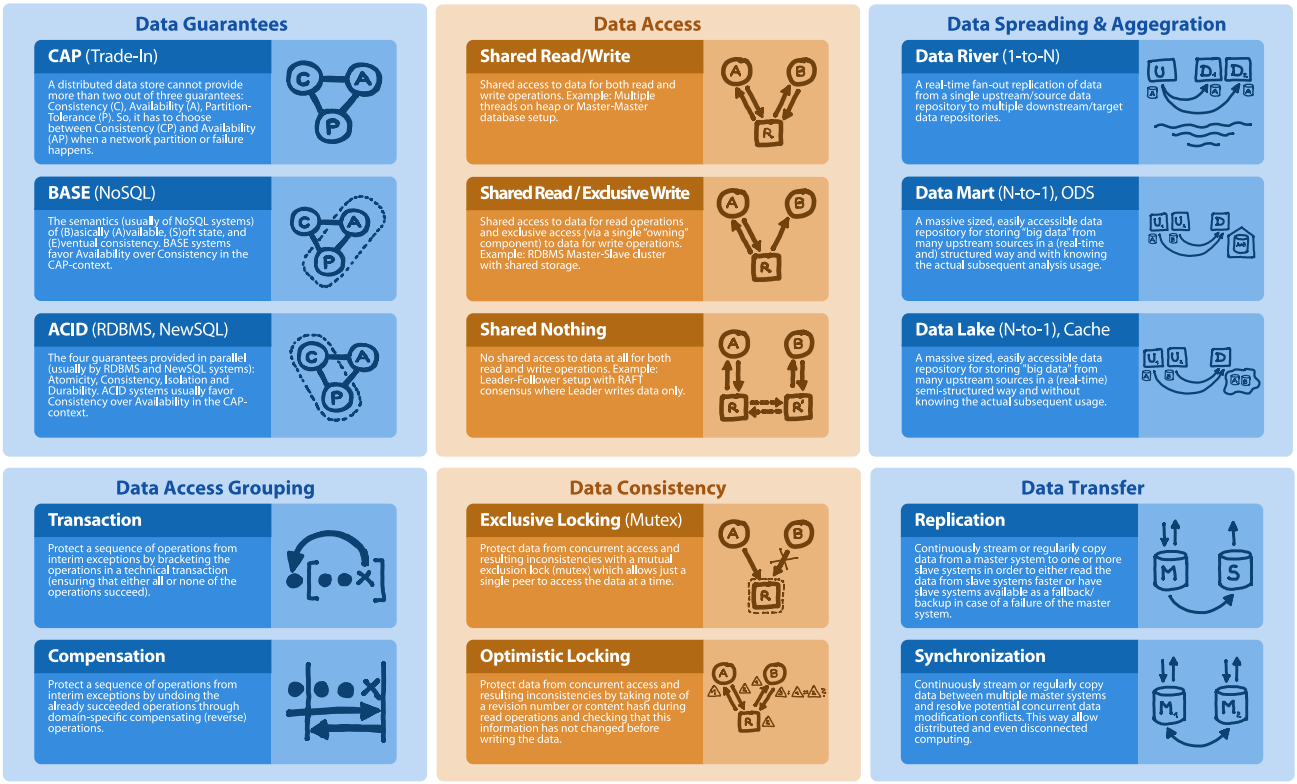
- Nennen Sie 3 **Data Evolution Approaches**, die es jeweils erlauben, auf die früheren Stände der Daten zuzugreifen?

ARCHITECTURE FUNDAMENTALS

Data Persistence Architectures

(Patterns)

TECHNISCHE UNIVERSITÄT MÜNCHEN



Intellectual Content: Version 1.0.2 (2015-04-22) Authored 2015-03-19 by Dr. Ralf S. Engelschall (rse@engelschall.com). Graphical Illustration: Version 1.1.0 (2022-04-14). Copyright © 2015-2022 Dr. Ralf S. Engelschall (http://engelschall.com). All rights reserved. Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Im Bereich der **Data Guarantees** gibt es drei wesentliche Aspekte: Das **CAP** Theorem adressiert den sog. "Trade-Off": Man muss sich in der Praxis üblicherweise zwischen Consistency + Partition-Tolerance (CP) oder Availability + Partition-Tolerance (AP) entscheiden. Beides gleichzeitig geht nicht. Bei **BASE**-Systemen favorisiert man üblicherweise AP. Bei einem traditionellen RDBMS mit **ACID** Garantien favorisiert man üblicherweise CP.

Beim **Data Access Grouping** kennt man **Transaction** und **Compensation**. Ersteres ist eine "technische Klammer", die einem erlaubt, im Fehlerfall wieder zu dem früheren Zustand zurückzukehren. Letzteres ist eine "fachliche Klammer", bei der sog. Kompensationsoperationen einem erlauben die früheren Änderungen zu "stornieren", um so einen früheren konsistenten Zustand wiederzuerlangen.

Beim **Data Access** von zwei oder mehr Prozessen/Threads auf die gleichen Daten unterscheidet man die Ansätze **Shared Read/Write** (alle lesen und schreiben dieselben Daten), **Shared Read / Exclusive Write** (alle lesen und nur einer schreibt dieselben Daten) und **Shared Nothing** (alle lesen und schreiben auf die gleichen synchronisierten Daten).

Bei der **Data Consistency** kennt man **Exclusive Locking** (pro Zeiteinheit schreibt nur einer) und **Optimistic Locking** (alle versuchen zu schreiben, erkennen und lösen aber einen Konflikt).

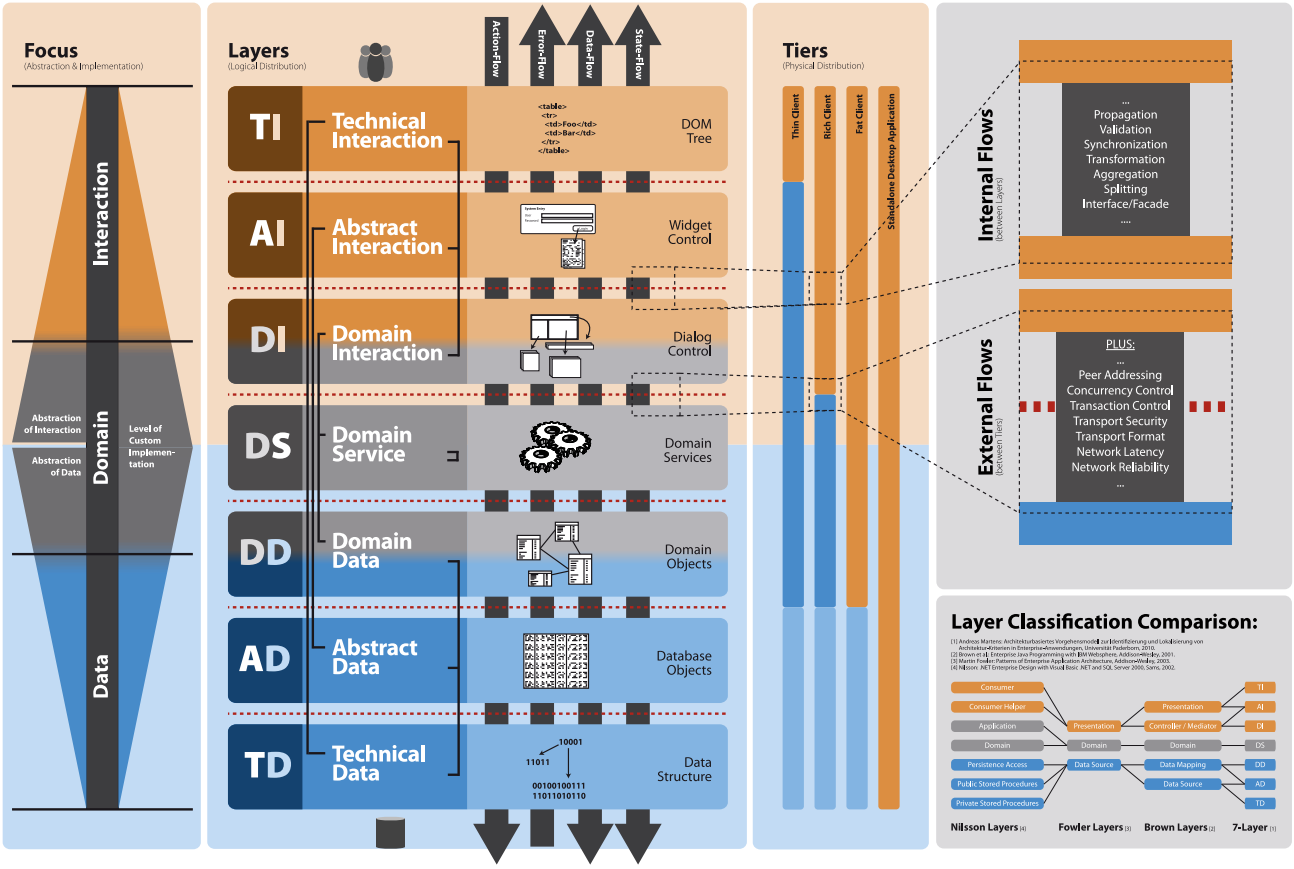
Beim **Data Spreading & Aggregation** unterscheidet man drei Arten: beim **Data River** werden die Daten von einem Master-System auf viele Slave-Systeme repliziert, um u.a. eine höhere Read-Performance zu erzielen. Beim **Data Mart** (strukturierte Daten) und **Data Lake** (semi-strukturierte Daten) werden dagegen die Daten von vielen Master-Systemen auf ein Slave-System repliziert, um u.a. die Daten zentral auswerten oder "cachen" zu können.

Beim **Data Transfer** wird zuletzt noch zwischen der unidirektionalen und konfliktfreien **Replication** und der bidirektionalen und konfliktreichen **Synchronisation** unterschieden.

Fragen

? Wie nennt man den Ansatz, bei dem Daten von einem Master-System auf viele Slave-Systeme repliziert werden?

Application Reference Architecture



In einer Anwendung kann man 7 logische Layer unterscheiden, die jeweils auf zwei Arten gruppiert sind: einerseits gibt es die drei sequenziellen Layer-Gruppen **Technical/Abstract/Domain Interaction**, **Domain Service** und **Domain/Abstract/Technical Data**, andererseits gibt es die drei geschachtelten Layer-Gruppen **Technical Interaction/Data**, **Abstract Interaction/Data** und **Domain Interaction/Service/Data**.

Zusätzlich kann man in einer Anwendung 4 primäre Flüsse unterscheiden: der **Action Flow** läuft konsequent nur von oben nach unten, da alle Aktionen oben vom Benutzer (oder Nachbarsystemen) angestoßen werden; der **Error Flow** läuft konsequent nur entsprechend in der Gegenrichtung, also von unten nach oben, da Fehler im Worst-Case dem Benutzer gemeldet werden müssen; der (fachliche) **Data Flow** und der (technische) **State Flow** laufen hingegen in beiden Richtungen, da Daten und Zustände sowohl persistiert als auch angezeigt werden müssen.

Die Abstraktion bei Interaction/Data in den Layern nimmt von oben/unten zur Mitte hin zu, weshalb auch dort der meiste fachliche Code einer Anwendung geschrieben wird. Für die oberen/unteren Layer wird üblicherweise massiv auf Open Source Libraries/Frameworks gesetzt.

Wenn man statt einem logischen Schnitt (der in einem **Internal Flow** zwischen den Layern resultiert) zwischen zwei Layern einen physikalischen Schnitt macht (der dann in einem **External Flow** resultiert), also die Anwendung in einzelne Programme auf unterschiedlichen Rechnern verteilt, so nennt man die daraus entstehende Architektur nach dem Umfang und der Verantwortung des Clients.

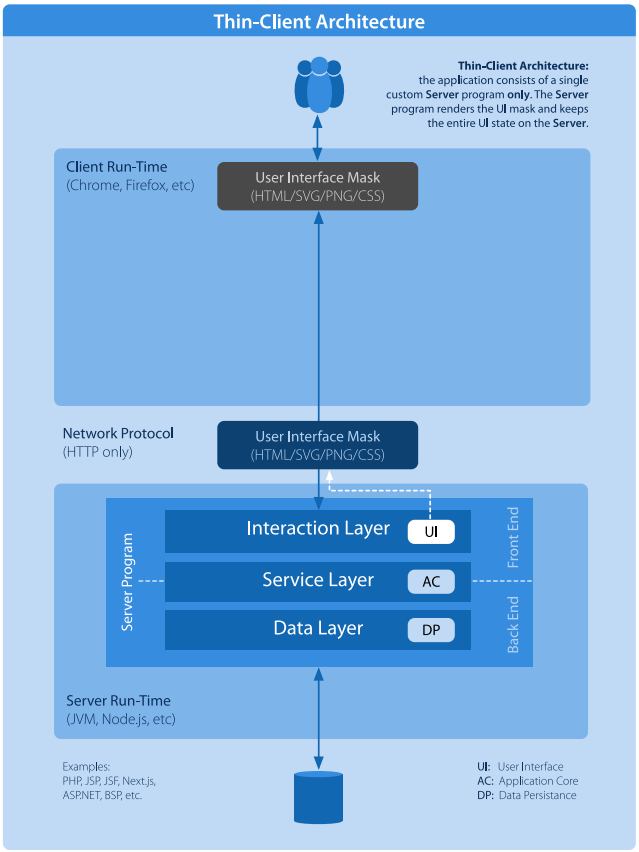
Bei **Thin Client** wird nur die **Technical Interaction** auf den Client ausgelagert, bei **Rich Client** wird die gesamte Benutzeroberfläche (also alle drei Layer **Technical/Abstract/Domain Interaction**) autonom auf den Client ausgelagert (üblicherweise als eine sog. "HTML5 Single-Page-Application"), bei **Fat Client** gibt es gar keinen zugehörigen Server mehr und bei der **Standalone** Application gibt es nur noch ein einziges Programm.

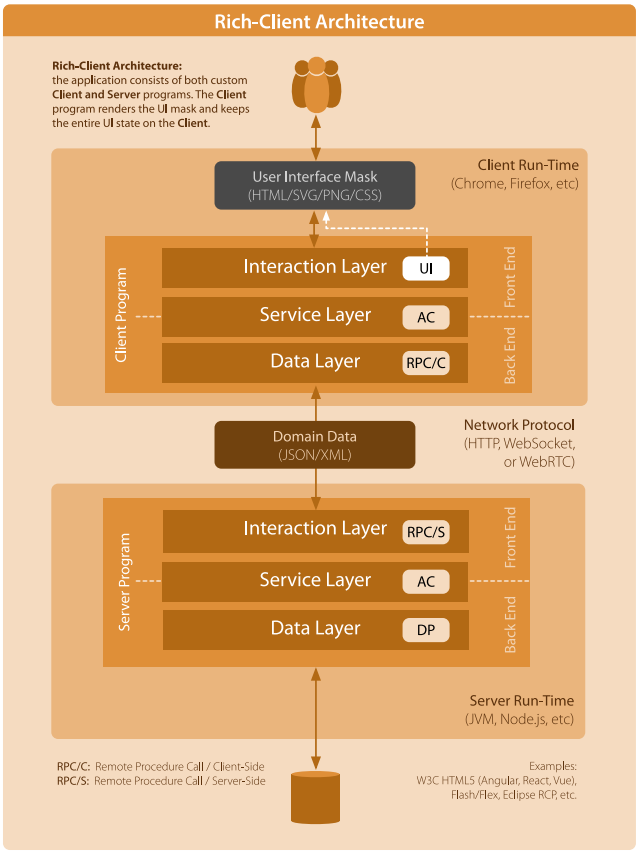
Fragen

- Wie nennt man die Anwendungs-Architektur, bei der die gesamte Benutzeroberfläche autonom auf dem Client läuft, während der Server nur rein fachliche Services liefert?
- Wie heißen die Web-Anwendungen, welche eine **Rich Client** Architektur umsetzen?

Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture comments only.

Client-Server Architecture





Bei der **Thin-Client-Architektur** besteht die Anwendung aus nur einem einzigen individuellen Server-Programm. Dieses Server-Programm erzeugt die Maske der Benutzeroberfläche und hält den gesamten Zustand der Benutzeroberfläche auf dem Server.

Der Vorteil dieser Architektur ist, daß die Anwendung sehr leicht aktualisiert werden kann. Der Nachteil dieser Architektur ist, daß die Benutzeroberfläche träge reagiert und der Zustand der Benutzeroberflächen aller Clients auf dem Server gehalten werden muss, was den Server zum Flaschenhals werden lassen kann.

Bei der **Rich-Client-Architektur** besteht die Anwendung sowohl aus einem individuellen Client- als auch einem Server-Programm. Das Client-Programm erzeugt die Maske der Benutzeroberfläche und hält den gesamten Zustand der Benutzeroberfläche auf dem Client.

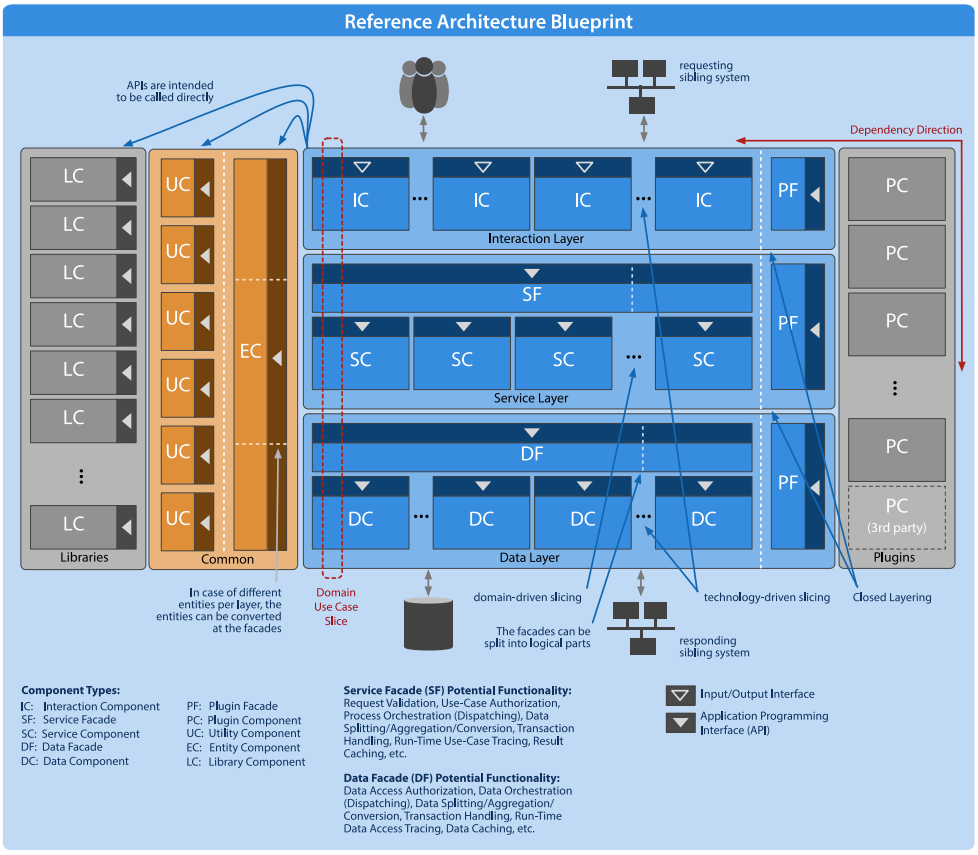
Der Vorteil dieser Architektur ist, daß die Benutzeroberfläche eine hohe Reaktionsfähigkeit zeigt, nur fachliche Daten zwischen Client und Server ausgetauscht werden müssen und der Server weniger stark zum Flaschenhals wird. Der Nachteil dieser Architektur ist, daß gegebenenfalls der Client explizit über eine Installationsprozedur aktualisiert werden muss.

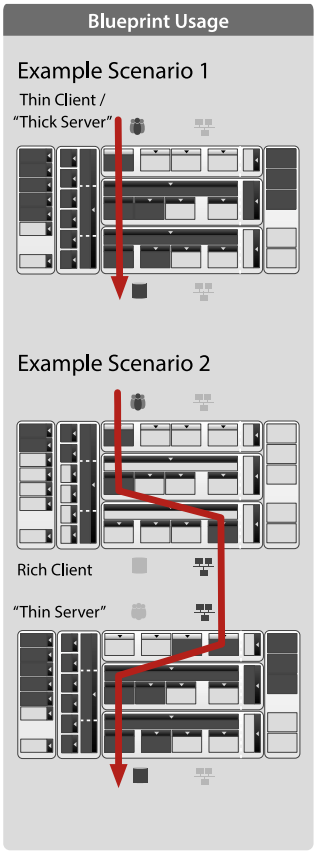
Fragen

- Bei welcher Client-Architektur bietet die Benutzeroberfläche die höhere Reaktionsgeschwindigkeit?

Intellectual Content: Version 1.0.0 (2021-04-14) Authored 2021 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.0 (2021-04-14) Copyright © 2021 Dr. Ralf S. Engelschall <http://engelschall.com> – All Rights Reserved.
Unauthorized Reproduction Prohibited. Licenced to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Information System Architecture





Ein (betriebliches) Informationssystem folgt üblicherweise einer stringenten Komponenten-basierten Referenz-Architektur. Diese wird "full blown" dargestellt und kann beliebig "abgespeckt" werden.

Zuerst besteht diese Referenz-Architektur aus 3 wesentlichen Layern: dem **Interaction Layer** mit den (technisch geschnittenen) Komponenten, welche die I/O-basierten Schnittstellen zum Benutzer (User Interface) und/oder anfragenden Nachbarsystemen (über Network Interface) bereitstellen, dem **Service Layer** mit den (fachlich geschnittenen) Service-Komponenten (auch Anwendungskern genannt) und dem **Data Layer** mit den (technisch geschnittenen) Komponenten, welche die Anbindung an die eigene Datenbank und/oder abzufragende Nachbarsysteme realisieren.

Man beachte, dass die "Andock-Position" eines Nachbarsystems von seinen Rollen abhängt: wenn es anfragt, dockt es am Interaction Layer an; wenn es abgefragt wird, dockt es am Data Layer an. Wenn es zufällig beide Rollen innehaben sollte, dockt es zweifach an. Die andere Sichtweise ist, dass sowohl der Benutzer als auch die Datenbank als spezielle "Nachbarsysteme" begriffen werden können.

Um die N **Interaction Components** (IC) mit M **Service Components** (SC) zu verbinden, wird üblicherweise eine entkoppelnde **Service Facade** (SF) eingezogen. Aus gleichem Grund gibt es üblicherweise auch eine **Data Facade**.

Das Datenmodell wird in gemeinsame **Entity Components** (EC) ausgelagert. Zusammen mit ggf. gemeinsam genutztem Code lebt beides in einem **Common Slice**. Libraries und Plugins sind ebenfalls in eigene Slices ausgelagert, es gibt aber zwei wesentliche Unterschiede: Libraries sind passiv und bieten der Anwendung ihre Funktionalität über ihre Schnittstellen an. Plugins sind aktiv und steuern die Anwendung, indem sie sich über Service-Provider Interfaces (SPI) in den **Plugin Facades** in die Anwendung einklinken.

In der Anwendung darf es nur genau eine **Dependency Direction** geben, damit die Anwendung (in der Gegenrichtung der Dependencies) sauber gebaut werden kann. Die Referenz-Architektur wird außerdem üblicherweise zweifach instanziiert, um sowohl einen Rich-Client als auch einen zugehörigen "Thin-Server" daraus zu konstruieren.

Fragen

- Mit welchem Layer-Pattern werden in einem Informationssystem die **Interaction Components** von den **Service Components** entkoppelt?
- In welcher Reihenfolge werden die Komponenten einer Anwendung gebaut?

Intellectual Content: Version 1.3.1 (2022-12-19), Authored 2002-2023 by Dr. Gernot Starke, Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.2.3 (2022-12-19), Copyright © 2017-2022 Dr. Ralf S. Engelschall (http://engelschall.com)
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in computer science lecture contexts only.